# MaaSAI Production Monitoring Tool

## Product Requirements Document (PRD)

| Field | Value |
|---|---|
| Document Version | 1.0 |
| Date | March 16, 2026 |
| Status | Draft — Awaiting Stakeholder Review |
| Product | MaaSAI Contract Monitoring & Production Visibility Tool |
| Owner | Engineering Team |
| Informed By | ARCHITECTURE.md · REQUEST.md · Case Studies P1, P2, P5 |

## Table of Contents


1. Problem Statement
2. Product Vision
3. User Personas
4. User Stories
5. Feature Requirements
6. Non-Functional Requirements
7. Success Metrics & KPIs
8. Pilot-Specific Requirements
9. Out of Scope
10. Assumptions & Dependencies
11. Open Questions
12. Acceptance Criteria Summary


## 1. Problem Statement

### The Current Pain (Before This Tool)

Manufacturing companies that outsource production to external providers today operate **blind**. The current reality across all three pilots is the same:

> **"During production, the sales team must repeatedly call or email providers to ask for status updates."** — Factor (Pilot 1) case study

This creates cascading problems:

| Problem | Impact |
|---|---|

| Problem | Impact |
|---|---|
| Manual status chasing (calls, emails) | Wastes hours per order per week across sales, ops, and quality teams |
| No visibility into production progress | Surprises at delivery — quality issues, delays discovered too late |
| Reactive quality control | Defects only found when parts arrive, no time to intervene |
| No audit trail | Disputes have no immutable record to reference |
| Siloed communication | Consumer and provider use different systems; no shared source of truth |
| Manual milestone verification | Approvals depend on emails; no structured workflow |

### The Opportunity

All three pilots (metal parts, gears, electronics/PCB) share the same monitoring pattern:

- A provider does work
- A consumer needs to see progress
- Key events need to be verified, approved, or disputed
- Everything needs to be recorded permanently

A **single generic monitoring tool**, configurable per manufacturing context, can solve this across all pilots and be extended to future ones.

---

## 2. Product Vision

**What we're building:**

> A real-time, blockchain-backed production monitoring platform that gives manufacturing consumers (buyers) full visibility into their outsourced production without requiring a single phone call or email.

**What success looks like:**

- A consumer opens their dashboard and immediately knows: what stage production is at, whether it's on track, what needs their attention, and what has been permanently verified.
- A provider pushes data from their factory floor — once — and that data flows automatically to the consumer, triggers alerts if something goes wrong, and logs key events immutably on the blockchain.
- Neither party needs to understand blockchain technology. It works invisibly, providing trust and auditability in the background.

**The north star metric:**

> **Zero manual status requests per order.** Every status update is automatic.

---

## 3. User Personas

### Persona 1 — The Consumer (Buyer / Ordering Company)

**Who they are:** A sales manager, procurement officer, or operations lead at a manufacturing company that has outsourced production to an external provider.

**Examples from pilots:**

- Factor's sales team monitoring an outsourced metal parts order
- ABC Manufacturing GmbH tracking a gear production order with Tasowheel
- A customer monitoring E4M's PCB development project

**What they need:**

- Know the current production status without asking anyone
- Be notified automatically when something needs their attention
- Approve or reject milestones with a single click
- Access documents and inspection reports
- Have confidence that what they see is accurate and tamper-proof

**What they don't want:**

- To see blockchain terminology (hashes, gas fees, block numbers)
- To manage technical integrations
- To wait for emails to know something went wrong

**Technical comfort:** Low to medium. Prefers dashboards over raw data.

---

### Persona 2 — The Provider (Manufacturer)

**Who they are:** The factory or manufacturing company executing the production order. They have machines, ERP systems, quality inspection tools, and edge data devices.

**Examples from pilots:**

- Factor's external metal parts supplier
- Tasowheel's production floor
- E4M's engineering and manufacturing team

**What they need:**

- A simple, reliable way to push production data from their systems
- Confirmation that data was received and processed
- Know which milestones are due and what evidence is required
- Receive alerts when they need to take action (e.g., consumer needs to approve before they can proceed)

**What they don't want:**

- Complex integrations that break
- Manually entering data into another portal
- Uncertainty about whether their data was received

**Technical comfort:** Medium. Ideally their existing ERP/MES pushes data automatically. Manual entry is a fallback.

---

### Persona 3 — The Platform Administrator

**Who they are:** An internal team member managing the platform — onboarding contracts, configuring pilot schemas, managing users.

**What they need:**

- Seed new contracts from blockchain agreement data
- Configure pilot-specific monitoring parameters per contract
- Monitor system health (which sensors are active, which contracts are stale)
- Access full blockchain audit logs for compliance

---

## 4. User Stories

### Consumer Stories

#### C1 — Contract Overview

> As a consumer, I want to see all my active manufacturing orders in one place, so I can quickly understand which ones need attention.

**Acceptance criteria:**

- List shows all contracts with status badge (On Track / Delayed / Action Required / Completed)
- Each row shows: product name, provider, delivery date, milestone progress (X/Y done)
- Contracts with `Action Required` status are highlighted at the top
- Page loads in < 2 seconds

---

#### C2 — Production Progress

> As a consumer, I want to see real-time production progress for an order without calling the provider.

**Acceptance criteria:**

- Dashboard shows current stage, quantity produced vs. planned, quality metrics
- Data is no older than the contract's configured `dataUpdateFrequency`
- A "Last updated X minutes ago" indicator is always visible
- If no update has been received within 2× the expected frequency, a warning is shown: "Provider data delayed"

---

#### C3 — Milestone Timeline

> As a consumer, I want to see all milestones on a visual timeline so I understand what has been completed and what is coming next.

**Acceptance criteria:**

- Timeline shows all milestones with planned dates and actual completion dates
- Each milestone has a clear status: Pending / In Progress / Submitted / Completed / Overdue
- Overdue milestones are highlighted in red
- Milestones with evidence attached show a document icon
- Clicking a milestone opens detail view with evidence files

---

### C4 — Milestone Approval

> As a consumer, when a milestone requires my approval, I want to be prompted to review and approve or reject it so production can continue.

**Acceptance criteria:**

- An "Action Required" badge appears on the contract when a milestone awaits approval
- The milestone detail view shows: completion evidence, provider notes, and Approve / Reject actions
- Approving records the decision with a timestamp
- Rejecting requires the consumer to enter a reason
- After action, blockchain records the event (visible in audit export)
- Provider is notified of the decision automatically

---

### C5 — Alert Awareness

> As a consumer, I want to receive clear alerts when something goes wrong during production so I can act early instead of discovering problems at delivery.

**Acceptance criteria:**

- Alerts are displayed with severity: Low / Medium / High / Critical
- Alert descriptions are in plain business language (not error codes)
- High and Critical alerts appear as a banner notification on the dashboard
- Consumer can acknowledge an alert (removes it from active list, keeps it in history)
- Alert history is always accessible with date/severity filters

---

### C6 — Document Access

> As a consumer, I want to access all documents related to an order (drawings, inspection reports, test results, certificates) in one place.

**Acceptance criteria:**

- Document list shows file name, type, linked milestone, upload date
- Documents can be downloaded
- Test result documents for E4M show pass/fail summary inline (no need to open the file)

---

### C7 — Analytics & KPI Tracking

> As a consumer, I want to see KPI metrics for my order so I can understand how performance compares to plan.

**Acceptance criteria:**

- Shows pilot-specific KPIs (defined in Section 7)
- Includes planned vs. actual timeline chart
- Includes quality trend over time (for applicable pilots)
- Data exportable to PDF or CSV

---

**C8 — Audit Export**

> As a consumer, I want to export a complete audit trail of an order for compliance and dispute resolution.

**Acceptance criteria:**

- Export includes all timeline events with timestamps
- Blockchain-verified events are marked as "Verified on blockchain"
- Export available as PDF
- Blockchain transaction references included in export (for auditors only, not in main UI)

---

## Provider Stories

**P1 — Data Push**

> As a provider, I want to push production updates from my factory systems to the platform so the consumer automatically stays informed.

**Acceptance criteria:**

- A single HTTP endpoint accepts production updates
- The endpoint validates data format and returns clear errors if data is malformed
- Successful submission returns confirmation including any alerts triggered
- Provider can push from their ERP, MES, or a manual form — all use the same endpoint

---

**P2 — Milestone Submission**

> As a provider, I want to mark a milestone as complete and attach evidence so the consumer can review and approve it.

**Acceptance criteria:**

- Provider can submit milestone completion via API with optional evidence file URLs
- Submission triggers consumer notification immediately
- If milestone is auto-verified (no consumer approval needed), it moves to Completed automatically
- If consumer approval is required, status moves to "Awaiting Approval"

---

**P3 — Feedback Visibility**

> As a provider, I want to know immediately when a consumer has approved or rejected a milestone so I know whether to proceed.

**Acceptance criteria:**

- Provider receives notification when consumer approves or rejects
- Rejection includes the consumer's stated reason
- Rejected milestone moves back to "In Progress" with rejection notes visible

---

### Administrator Stories

**A1 — Contract Onboarding**

> As an admin, I want to register a new contract from a blockchain agreement so monitoring begins automatically.

**Acceptance criteria:**

- Admin can create a contract record by providing the blockchain contract address
- System pulls agreement metadata from blockchain
- Milestones are seeded from the agreement's schedule
- Alert rules are configured from the agreement's `alertConditions` array
- Contract status begins as `PENDING_SIGNATURES` or `ACTIVE` depending on blockchain state

---

**A2 — System Health**

> As an admin, I want to see which provider connections are active and which contracts have gone stale.

**Acceptance criteria:**

- Admin view shows last-received update timestamp per contract
- Contracts with no update in > 2× expected frequency flagged as "Stale"
- Admin can manually trigger an alert if needed

---

## 5. Feature Requirements

### F1 — Provider Ingest API

| # | Requirement | Priority |
|---|---|---|
| F1.1 | Generic HTTP POST endpoint accepting updates from any pilot type | Must Have |
| F1.2 | Payload validated against registered JSON schema for the contract's pilot type | Must Have |
| F1.3 | Returns structured response: `updateId`, `processed`, `alertsTriggered`, `milestoneUpdated` | Must Have |
| F1.4 | Bearer token authentication per provider | Must Have |

| # | Requirement | Priority |
|---|---|---|
| F1.5 | Support `updateType` enum: `PRODUCTION_UPDATE`, `MILESTONE_COMPLETE`, `QUALITY_EVENT`, `PHASE_CHANGE`, `DOCUMENT_UPLOAD` | Must Have |
| F1.6 | New pilot types registered by adding a JSON schema file — no code changes | Must Have |
| F1.7 | Rate limiting (configurable per organisation) | Should Have |
| F1.8 | Idempotency key support (prevent duplicate updates) | Should Have |

### F2 — Contract State Management

| # | Requirement | Priority |
|---|---|---|
| F2.1 | Contracts follow the defined state machine: `PENDING_SIGNATURES → ACTIVE → IN_PROGRESS → AWAITING_DELIVERY → COMPLETED / DISPUTED → CLOSED` | Must Have |
| F2.2 | State transitions written to PostgreSQL immediately | Must Have |
| F2.3 | State transitions queued for async blockchain logging | Must Have |
| F2.4 | State transitions trigger consumer notifications | Must Have |
| F2.5 | Dispute state initiates escalation notification to both parties | Must Have |

### F3 — Milestone Engine

| # | Requirement | Priority |
|---|---|---|
| F3.1 | Milestones defined per contract with: `name`, `plannedDate`, `completionCriteria`, `approvalRequired` | Must Have |
| F3.2 | Milestone state machine: `PENDING → IN_PROGRESS → SUBMITTED → VERIFIED → APPROVED → COMPLETED / REJECTED` | Must Have |
| F3.3 | Auto-verification: system checks completion criteria against incoming data | Must Have |
| F3.4 | Manual approval flow: consumer approves or rejects with reason | Must Have |
| F3.5 | Milestone completion logged on blockchain | Must Have |

| # | Requirement | Priority |
|---|---|---|
| F3.6 | Evidence files (documents, test results) attached to milestones | Must Have |
| F3.7 | Overdue milestone detection (planned date passed, not completed) | Must Have |

### F4 — Alert & Rule Engine

| # | Requirement | Priority |
|---|---|---|
| F4.1 | Rule engine evaluates every incoming update against configured alert conditions | Must Have |
| F4.2 | Built-in rule types: `DELAY`, `QUALITY_THRESHOLD`, `NO_DATA_RECEIVED`, `TEST_FAILURE`, `MILESTONE_OVERDUE` | Must Have |
| F4.3 | Alert conditions configurable per contract from agreement `alertConditions` JSON | Must Have |
| F4.4 | Alert severities: `LOW`, `MEDIUM`, `HIGH`, `CRITICAL` | Must Have |
| F4.5 | `HIGH` and `CRITICAL` alerts logged on blockchain | Must Have |
| F4.6 | Consumer notified on alert creation | Must Have |
| F4.7 | Consumer can acknowledge alerts | Must Have |
| F4.8 | Alert history permanently retained | Must Have |
| F4.9 | No-data-received alert fires after 2× expected update frequency with no update | Must Have |

### F5 — Consumer Dashboard (UI)

| # | Requirement | Priority |
|---|---|---|
| F5.1 | Contracts list page with status badges and quick stats | Must Have |
| F5.2 | Contract overview page: status banner, progress bar, key details, next milestone countdown | Must Have |
| F5.3 | Milestone timeline page: visual timeline with status icons, approval actions, evidence access | Must Have |
| F5.4 | Production feed page: live metrics updated via WebSocket, pilot-adaptive layout | Must Have |

| # | Requirement | Priority |
|---|---|---|
| F5.5 | Alert center page: active alerts, severity badges, acknowledge action, history | Must Have |
| F5.6 | Documents page: all files linked to contract, downloadable | Must Have |
| F5.7 | Analytics page: pilot-specific KPIs and charts | Should Have |
| F5.8 | Audit export: full timeline as PDF with blockchain verification markers | Should Have |
| F5.9 | Zero blockchain terminology in any consumer-facing screen | Must Have |
| F5.10 | All blockchain terms translated to plain business language | Must Have |
| F5.11 | Real-time updates via WebSocket — no manual page refresh required | Must Have |
| F5.12 | Offline provider warning when data feed goes stale | Must Have |
| F5.13 | Mobile-responsive layout | Should Have |

### F6 — Blockchain Integration

| # | Requirement | Priority |
|---|---|---|
| F6.1 | Swappable blockchain adapter: `mock` (local dev) and `real` (production) via env var | Must Have |
| F6.2 | `real` adapter connects to `https://entrynet-maasai.euinno.eu` via `web3.py` | Must Have |
| F6.3 | Events written to blockchain: contract activation, milestone completion, HIGH/CRITICAL alerts, dispute events, closure | Must Have |
| F6.4 | Explorer links use `https://blockscout-maasai.euinno.eu/tx/{txHash}` | Must Have |
| F6.5 | Blockchain writes are async (do not block API response) | Must Have |
| F6.6 | Failed blockchain writes are retried with exponential backoff | Must Have |
| F6.7 | MetaMask transaction signing flow for consumer milestone approvals (optional, v2) | Nice to Have |

### F7 — Mock Sensor Infrastructure (Dev / Testing)

| # | Requirement | Priority |
|---|---|---|
| F7.1 | Simulator per pilot: Factor, Tasowheel, E4M | Must Have |

| # | Requirement | Priority |
|---|---|---|
| F7.2 | Configurable scenario per simulator via env var | Must Have |
| F7.3 | Scenarios: `normal`, `delay`, `quality_failure`, `milestone_complete`, `dispute` | Must Have |
| F7.4 | Scenario-driven event sequences (scripted data, not purely random) | Must Have |
| F7.5 | Configurable push interval per simulator | Must Have |
| F7.6 | Simulators ship as Docker containers in `docker-compose.dev.yml` | Must Have |

## 6. Non-Functional Requirements

### Performance

| Requirement | Target |
|---|---|
| API response time (read endpoints) | < 200ms p95 |
| API response time (ingest endpoint) | < 500ms p95 |
| WebSocket message delivery latency | < 1 second from event to UI |
| Dashboard initial load | < 2 seconds |
| Database query time for timeline (last 100 events) | < 100ms |

### Reliability

| Requirement | Target |
|---|---|
| API availability | 99.5% uptime (dev/staging target) |
| Ingest endpoint — no data loss | Failed writes retried; not dropped |
| Blockchain writes — eventual consistency | Events written within 60 seconds of trigger |
| Alert delivery | 100% — no alert silently dropped |

### Security

| Requirement | Details |
|---|---|
| Provider authentication | Bearer token per provider; tokens scoped to specific contract(s) |
| Consumer authentication | Session-based auth; consumers can only see their own contracts |
| Data in transit | HTTPS only; WebSocket over WSS |
| GDPR compliance | Personal data (names, contact details) not stored on blockchain; only anonymised IDs |

| Requirement | Details |
|---|---|
| Sensitive commercial data | Stored off-chain only; blockchain stores hashes |

### Extensibility

| Requirement | Details |
|---|---|
| New pilot onboarding | Adding a new pilot requires only a new JSON schema file — no backend code changes |
| New alert rule types | Rule engine supports adding new rule types via configuration |
| New KPI metrics | Analytics service supports new metrics via configuration per pilot type |

### Observability

| Requirement | Details |
|---|---|
| Structured logging | All API requests and service events logged in JSON format |
| Blockchain sync status | Admin can see which events are pending, confirmed, or failed |
| Mock sensor health | Each simulator reports last push time and active scenario |

## 7. Success Metrics & KPIs

### Product-Level Success Metrics

| Metric | Baseline (Before) | Target (After) |
|---|---|---|
| Manual status requests per order | Multiple calls/emails per order | **Zero** |
| % of production updates that are automated | ~0% (all manual) | **100%** |
| Time to detect a production issue | Days (discovered at delivery) | **Hours (during production)** |
| Time to close a quality non-conformity | Days | **< 24 hours** |
| Milestone approval turnaround | Days (email chain) | **Same day** |

### Pilot 1 — Factor KPIs

| KPI ID | Name | Measurement Method |
|---|---|---|
| KPI1_P1_BP02 | Average manual status requests per order | Count of calls/emails per order — target: 0 |

| KPI ID | Name | Measurement Method |
|---|---|---|
| KPI2_P1_BP02 | Customer satisfaction with order status updates | Survey score — target: increase |
| KPI3_P1_BP02 | % of automated status updates | `automated_updates / total_updates × 100` — target: 100% |
| KPI1_P1_BP03 | Proactive quality issue detection rate | Issues found during monitoring / total issues |
| KPI2_P1_BP03 | Average time to close quality non-conformities | Days from detection to resolution |

### Pilot 2 — Tasowheel KPIs

| KPI ID | Name | Measurement Method |
|---|---|---|
| KPI1_P2_BP01 | Effective resource utilisation | Actual production time / available time |
| KPI3_P2_BP01 | Manufacturing process capability (OTD) | On-time delivery rate |
| KPI4_P2_BP01 | Downtime duration | Total downtime minutes per order |
| KPI5_P2_BP01 | Production efficiency index | Planned vs. actual cycle time ratio |
| KPI7_P2_BP01 | Carbon footprint per product | $kgCO_2e$ per piece (from `carbonKgCo2e` field) |
| KPI8_P2_BP01 | Energy consumption | kWh per order (from `energyKwh` field) |

### Pilot 5 — E4M KPIs

| KPI ID | Name | Measurement Method |
|---|---|---|
| KPI1_P5_BP02 | Manufacturing Time Availability Gain | Actual phase duration vs. planned |
| KPI2_P5_BP02 | Production Flow Efficiency Improvement | Phase transitions per unit time |
| KPI3_P5_BP02 | Waste Avoidance Through Remanufacturing | Defect rate leading to rework |

## 8. Pilot-Specific Requirements

### Pilot 1 — Factor (Metal Parts)

**Monitoring focus:** Quantity-based production progress and quality pass rate.

| Requirement | Details |
|---|---|
| Update type | Quantity produced, quality pass rate, machine utilisation, current stage |

| Requirement | Details |
|---|---|
| Update frequency | Configurable per contract; typically every 4 hours |
| Alert: No data received | Fire after 2× expected frequency with no update |
| Alert: Quality threshold | Fire when `qualityPassRate` drops below contract-configured threshold (default 0.95) |
| Alert: Production delay | Fire when production progress falls behind schedule by > configured days |
| Consumer view | Progress bar (qty produced / planned), quality gauge, stage indicator |
| Milestone type | Production stages (turning, heat treatment, grinding, inspection) |
| Approval gate | Final quality acceptance at delivery |

### Pilot 2 — Tasowheel (Gears)

**Monitoring focus:** Manufacturing routing step completion, efficiency, and sustainability metrics.

| Requirement | Details |
|---|---|
| Update type | Routing step name/status, setup time, cycle time, downtime, energy kWh, carbon kg$CO_2$e |
| Update frequency | Event-driven — one push per routing step completion |
| Alert: Downtime | Fire when `downtimeMinutes` exceeds contract-configured threshold |
| Alert: Cycle time deviation | Fire when actual cycle time deviates from planned by > X% |
| Alert: Routing step delay | Fire when step completion date passes without update |
| Consumer view | Routing steps checklist, efficiency %, energy and carbon running totals |
| Milestone type | Manufacturing routing steps (turning → heat treatment → grinding → inspection) |
| Approval gate | Order acceptance (pre-production); final delivery acceptance |
| Special requirement | Carbon and energy data must be captured per order for KPI7 and KPI8 |

### Pilot 5 — E4M (Electronics/PCB)

**Monitoring focus:** Development phase progress, test results, and customer approval gates.

| Requirement | Details |
|---|---|
| Update type | Current phase (M1–M6), completion %, deliverables, test results, open issues |

| Requirement | Details |
|---|---|
| Update frequency | Phase-level — updates when phase progress changes (days apart) |
| Alert: Phase delay | Fire when phase passes planned completion date without completion |
| Alert: Test failure | Fire immediately when any `testResults` entry has `result: FAIL` |
| Alert: Approval overdue | Fire when `approvalRequired: true` but no consumer response within N days |
| Alert: Critical issue | Fire when open issue with severity `HIGH` or `CRITICAL` is submitted |
| Consumer view | Phase progress pipeline (M1→M6), test results table, deliverables list |
| Milestone type | M1 Idealization → M2 Concept → M3 Engineering → M4 Prototype → M5 Testing → M6 Production |
| Approval gates | **Required:** M2 (concept), M3 (engineering), M5 (testing), M6 (production batch) |
| Special requirement | Project type field: `DesignAndManufacture` vs `ManufactureOnly` (affects which milestones apply) |
| Special requirement | PCB test types: AOI, Flying Probe, Functional — each has distinct pass/fail tracking |

## 9. Out of Scope

The following are explicitly **not** part of this tool. They are handled by other MaaSAI components:

| Feature | Handled By |
|---|---|
| Smart contract creation and deployment | SCT (Smart Contracts Toolkit) |
| Agreement negotiation and quotation | MCH (MaaS Collaboration Hub) / CMM (Cloud MaaS Marketplace) |
| Provider search and matching | MaaS Dynamic Catalogue / Recommendation System |
| Blockchain infrastructure management | MSB team (SLG) |
| User identity management / SSO | Separate auth service (TBD) |
| ERP/MES direct integration | Data Integrator component (not this tool) |
| E4M Maintenance monitoring (BP03) | Future scope — PMT Predictive Maintenance Toolkit |
| E4M End-of-life management (BP04) | Future scope — R3T Toolkit |
| Payment processing or triggering | SCT automated payment triggers |
| Mobile native app | Web app only (mobile-responsive) |

# 10. Assumptions & Dependencies

## Assumptions

| # | Assumption |
|---|---|
| A1 | The blockchain agreement (contract address) already exists before monitoring begins. This tool monitors — it does not create contracts. |
| A2 | Provider authentication tokens are issued and managed externally (not within this tool for v1). |
| A3 | File storage for evidence documents (drawings, test reports) is handled by an external service (S3-compatible or IPFS). This tool stores references (URLs), not files. |
| A4 | The consumer UI is only accessible to authenticated users. Auth mechanism is out of scope for v1 but the backend is auth-ready. |
| A5 | For local development, all three mock sensors run simultaneously against pre-seeded contracts. |
| A6 | The `dataUpdateFrequency` configured in the smart contract agreement is the source of truth for how often the provider is expected to push data. |

## Dependencies

| Dependency | Status | Risk |
|---|---|---|
| MSB blockchain network live (`https://entrynet-maasai.euinno.eu`) | Live (SLG team) | Low — mock adapter covers dev |
| Smart contract ABI for manufacturing monitoring contracts | Not yet defined — current MVP is Barba Stathis pilot | **Medium** — needed for production blockchain integration |
| Agreement JSON schema v1.0 (from REQUEST.md) | Draft — pending pilot validation | **Medium** — mock data can proceed without it |
| Provider Edge devices / ERP APIs | Not connected in v1 (mock sensors substitute) | Low for v1 |
| File storage service (S3/IPFS) | TBD | Low for v1 (document URLs are strings) |

# 11. Open Questions

| # | Question | Impact | Owner |
|---|---|---|---|
| OQ1 | Which MaaSAI tool reads 3D drawings/STEP/IGES files? (Tasowheel open question) | Affects `technicalDrawings` parameter handling | MaaSAI Platform Team |

| # | Question | Impact | Owner |
|---|---|---|---|
| OQ2 | What is the final smart contract ABI for manufacturing monitoring? Current MVP is Barba Stathis (food). | Blocks production blockchain integration | SLG Blockchain Team |
| OQ3 | How are provider bearer tokens issued and rotated? | Blocks provider authentication design | Platform / Auth Team |
| OQ4 | What is the agreed `dataUpdateFrequency` per pilot? (Factor: every 4 hours confirmed; Tasowheel and E4M TBC) | Affects no-data alert thresholds | Pilot Teams |
| OQ5 | Does the consumer need a dedicated mobile app, or is mobile-responsive web sufficient? | Scope and effort | Stakeholders |
| OQ6 | Should MetaMask signing be required for consumer milestone approvals in v1, or is session-based auth sufficient? | Blockchain integration complexity | SLG + Engineering |
| OQ7 | What file formats are valid for evidence attachments? (PDF confirmed; are CAD files, Gerber files, etc. also required?) | Document handling requirements | Pilot Teams |
| OQ8 | Are there SLA requirements for the monitoring API (uptime, support hours)? | Infrastructure and ops scope | Project Management |

## 12. Acceptance Criteria Summary

The tool is considered **feature-complete for v1** when all of the following are true:

### Backend

- `POST /api/v1/ingest/{contractId}` accepts and validates payloads for all 3 pilot types
- Rule engine fires correct alerts for all defined alert conditions across all 3 pilots
- Milestone state machine correctly transitions through all states including approval and rejection
- All consumer read endpoints return correct data
- WebSocket delivers updates to connected clients within 1 second of an ingest event
- `BLOCKCHAIN_ADAPTER=mock` logs events to local SQLite; `=real` connects to `entrynet-maasai.euinno.eu`
- Blockchain writes are async and do not block API responses
- Failed blockchain writes are retried

### Mock Sensors

- Factor simulator generates valid payloads and pushes at configured interval
- Tasowheel simulator generates valid payloads with energy and carbon fields
- E4M simulator generates phase updates, test results, and deliverables
- All 3 simulators support `normal` and at least one failure scenario each

- `docker compose -f docker-compose.dev.yml up` starts all services with no errors and data flows

### Frontend

- Contracts list shows all seeded contracts with correct status badges
- Contract overview shows live production metrics adapting to pilot type
- Milestone timeline shows correct statuses and allows consumer to approve/reject where required
- Alert center shows all triggered alerts; acknowledge action works
- Production feed receives live WebSocket updates without page refresh
- No blockchain terminology visible anywhere in the consumer UI
- Approval of a milestone records the event and notifies the provider

### Cross-Cutting

- The same frontend and backend serve all 3 pilot types without code branching in the UI layer
- Adding a 4th pilot type requires only: a new JSON schema file + seed data — no code changes
- All API responses are consistently structured (envelope pattern with `data`, `error`, `meta`)

---

## Appendix A — Blockchain Term Translation Reference

| Raw Blockchain Term | What the Consumer UI Shows |
|---|---|
| Smart contract deployed on MSB | "Agreement registered" |
| Contract state: `ACTIVE` | "In Progress" |
| Milestone logged on-chain | "Milestone confirmed and recorded" |
| Transaction hash | Hidden (audit export only) |
| Block number | Hidden |
| Gas fee | Hidden |
| Wallet signature | "Approved by [Name]" |
| Contract address | Hidden (admin export only) |
| Dispute on-chain | "Issue raised — under review" |

---

## Appendix B — Contract State Machine Reference

```
PENDING_SIGNATURES  ──► ACTIVE  ──► IN_PROGRESS  ──► AWAITING_DELIVERY
                                                          │

┌──────────────┴──────────────┐
                                                  ▼
▼
                                            COMPLETED
DISPUTED
```

```
                                                    |
  |                                                 ▼
  ▼                                               CLOSED
RESOLVED ──► CLOSED
```

---

## Appendix C — Pilot Type Quick Reference

| Field | Factor (P1) | Tasowheel (P2) | E4M (P5) |
|---|---|---|---|
| `pilotType` | `FACTOR` | `TASOWHEEL` | `E4M` |
| Industry | Metal machining | Gear manufacturing | PCB / Electronics |
| Country | Spain | Finland | Portugal |
| Primary update | Production qty + quality | Routing step + efficiency | Development phase + tests |
| Milestone model | Production stages | Routing steps | M1–M6 phases |
| Approval gates | Quality acceptance | Order acceptance | M2, M3, M5, M6 |
| Special fields | Machine utilisation | Energy, carbon, downtime | Test types, deliverables, issues |
| Update frequency | Every 4 hours | Per step event | Per phase change |